YUVA INTERN

EMBEDDED SYSTEMS INTERNSHIP | WEEK 2 TECHNICAL REPORT

IoT-Based Smart Environmental Monitoring and Alert System

Interrupt-driven ESP32 firmware, implemented and tested on a simulated embedded system

Week 2 Task

Firmware Development and Implementation

Item	Details
Student Name	Pilla Yashwanth (Yuva Intern)
Email	Yaswanthvasu79@gmail.com
Institute	Ignite Embedded Systems
Internship Organization	Yuva Intern
Report Date	14/10/2026
Deliverables	smart_env_monitor.c (single-file firmware), README.md, test_results.txt, demo_output.txt, this report
Test Platform	Simulated hardware (PC, GCC). No physical hardware was used in Week 2.

Table of Contents

1. Introduction 4
2. Objectives and Scope 4
2.1 Out of Scope for Week 2 4
3. Selected Firmware Application 4
4. Development Approach and Environment 5
4.1 Why a Simulated Embedded System 5
4.2 What Is Simulated and What Is Not 5
4.3 Tools 5
4.4 Development Process 5
5. Firmware Architecture 6
5.1 Modules 7
5.2 State Machine 7
5.3 Shared Data Structures 8
5.4 HAL Interface 8
6. Interrupt Design 8
6.1 Timer Interrupt 8
6.2 UART Receive Interrupt and Ring Buffer 9
6.3 Why the Main Loop Never Blocks 9
7. Key Algorithms 9
7.1 Validation Pipeline for DHT22 Samples 9
7.2 MQ-135 Processing: Relative Index 9
7.3 Hysteresis and Confirmation 9
7.4 Wi-Fi State Machine and Back-off 10
7.5 Telemetry Message 10
7.6 Display Update Strategy 10
8. Configuration Parameters 10
9. UART Command Interface 11
10. Requirement Implementation Matrix 11
11. Test Strategy 12
12. Test Cases and Results 13
12.1 Values Observed in the Simulation 14
12.2 Example Scenario Log 15
13. Debugging and Defect Log 15
14. Code Quality and Documentation 16
15. How to Compile and Run 16
16. Porting to ESP32 Hardware 16
17. Limitations and Known Issues 17
18. Week 2 Deliverables 17
19. Preparation for Weeks 3 and 4 18

20. Conclusion 18
Appendix A: Self-Test Output (test_results.txt) 19
Appendix B: Source File Map 19

1. Introduction

Week 1 produced the requirements and architecture of an ESP32-based environmental monitoring and alert node. Week 2 turns that design into working firmware. The task is to design and implement firmware on a **simulated** embedded system, with thorough inline documentation, a README, and a documented test process.

The firmware is delivered as **one C file**, `smart_env_monitor.c`. It contains the complete firmware logic, a simulated hardware layer, and a self-test suite of 20 tests. It compiles with a standard C99 compiler and runs on a PC, so every behaviour in this report can be reproduced by compiling and running one file.

> **Scope and honesty statement.** All testing in this report was done on a PC simulation. No ESP32, sensor, OLED, buzzer or Wi-Fi network was used. Timing values are simulated time, not measurements on real hardware. Porting to ESP32 hardware is described (Section 16) but was not done or tested in Week 2. Thresholds, the air-quality percentages and the MQ-135 warm-up time are initial proposals.

The report follows the structure expected for the task: application selection (Section 3), firmware architecture and interrupts (Sections 5 and 6), implementation details (Sections 7 to 9), requirement mapping (Section 10), the testing process and results (Sections 11 to 13), code quality and compile/run instructions (Sections 14 and 15), and limitations, deliverables and preparation for Weeks 3 and 4 (Sections 17 to 19).

2. Objectives and Scope

ID	Objective
W2-1	Implement the Week 1 design as firmware: sensing, processing, display, threshold monitoring, alerting, Wi-Fi telemetry and error handling.
W2-2	Demonstrate hardware-oriented techniques: a timer interrupt, a UART interrupt with a ring buffer, and a UART command interface.
W2-3	Keep the firmware hardware-independent behind a hardware abstraction layer (HAL), and simulate the hardware so the code can be tested without a board.
W2-4	Test systematically with fault injection, and record the process, results and defects honestly.
W2-5	Document the code with inline comments and a README that explains how to compile, run and port it.

2.1 Out of Scope for Week 2

- Running on physical ESP32 hardware, current or timing measurements on hardware (planned for Weeks 3 and 4).
- A real cloud service or dashboard; telemetry is produced and delivered to a simulated Wi-Fi layer only.
- Calibration of the MQ-135 or tuning of thresholds against real environments.
- Performance optimisation and integration testing (Weeks 3 and 4).

3. Selected Firmware Application

The firmware application is the complete control software of the Week 1 system, the **IoT-Based Smart Environmental Monitoring and Alert System**. It combines several of the example applications named in the task (timer-interrupt-driven scheduling, UART communication and sensor data reading) in one coherent embedded application, which is why it was chosen over a single-feature example such as an LED blinker.

Table 1: How the task's example applications appear in this firmware

Example in the task	Where it appears in this firmware
LED control using timer interrupts	timer_isr_1ms() generates the blink phases that drive alert_task(): steady LED and 250 ms buzzer beeping for an alert, 1 s slow blink for a fault.
UART communication	uart_rx_isr() and cli_poll() implement an interrupt-driven command interface (STATUS, THRESH, SET, MUTE, HELP) and the diagnostic log.

Example in the task	Where it appears in this firmware
Sensor data reading	dht_task() and mq_task() acquire, validate and filter DHT22 and MQ-135 data.

The Week 1 requirements (FR-01 to FR-26, NFR-01 to NFR-13) are the specification for this work. Section 10 maps each functional requirement to the function that implements it and the test that verifies it.

4. Development Approach and Environment

4.1 Why a Simulated Embedded System

The task allows a simulated system. Rather than using a graphical circuit simulator, the design uses a **software simulation of the hardware boundary**. The firmware calls only a small set of HAL functions (for example hal_dht_read, hal_oled_write_frame, hal_wifi_send). In the simulation these functions are implemented by a simulated board that can be controlled and observed from a test. This approach has three benefits: the exact firmware logic that would run on the ESP32 is tested, faults that are awkward to create on a bench (a failing Wi-Fi network, a stuck ADC, a lock-up) can be injected on demand, and tests run in virtual time, so a 120-second scenario completes in milliseconds and gives identical results every time.

4.2 What Is Simulated and What Is Not

Table 2: Simulation scope

Element	How it is simulated	What the simulation does not capture
System clock and timer interrupt	A virtual millisecond clock calls timer_isr_1ms() once per tick, then runs one pass of fw_loop().	Real interrupt latency, preemption of the main loop, race conditions.
DHT22	Returns injected temperature and humidity, or no response.	Real protocol timing, checksum errors, self-heating, accuracy.
MQ-135 and ADC	Returns an injected 12-bit value plus small deterministic noise (3 counts).	Real sensor dynamics, drift, ADC non-linearity, temperature/humidity dependence.
OLED	Stores the last frame; can print it as a box.	I2C timing and bus errors.
LED and buzzer	Boolean outputs; time-on and toggle count are recorded.	Electrical behaviour.
Wi-Fi	Connection delay, access-point availability, send success are controlled by the test.	Real network latency, packet loss, signal strength.
UART	Output captured to a buffer; input delivered byte by byte (about 1 byte per ms) through the receive interrupt.	Baud-rate errors, framing errors.
Watchdog and resets	Expires if not fed for 10 s; reboots the firmware with a reset cause.	Real hardware reset behaviour.

4.3 Tools

- **Language and compiler:** C99, GCC, compiled with -Wall -Wextra -Wpedantic. No external libraries.
- **Checking tools:** AddressSanitizer and UndefinedBehaviorSanitizer builds of the same file, used to look for memory errors and undefined behaviour while the tests and demo run.
- **Test harness:** built into the same file (section 10 of the source), so the deliverable remains a single file.

4.4 Development Process

Table 3: Methodical development process followed in Week 2

No.	Step	Description	Result
1	Plan from Week 1	Took the module list, state model, thresholds and timing targets from the Week 1 report as the specification.	Section 5
2	Define the HAL boundary	Listed every hardware interaction as a HAL function before writing logic.	Section 5.4

No.	Step	Description	Result
3	Implement interrupts and scheduler	Timer ISR, UART ISR, flags and the cooperative main loop.	Section 6
4	Implement modules incrementally	Acquisition and validation, thresholds, state machine, alerts, display, Wi-Fi, CLI.	Section 7
5	Build the simulator	Simulated board with injection and observation points.	Section 4.2
6	Write tests per requirement	20 tests covering normal operation, alerts and every fault in the Week 1 fault table.	Section 12
7	Run, observe, fix	Ran the suite, analysed failures, and corrected firmware or test as appropriate.	Section 13
8	Check robustness	Strict warnings at two optimisation levels, plus sanitizer builds.	Section 14
9	Document	Inline comments, embedded README, README.md, this report, captured logs.	Section 18

5. Firmware Architecture

The firmware keeps interrupt work and main-loop work strictly apart. Interrupts only keep time, raise flags and store received bytes. All processing happens in a cooperative main loop that never blocks. Hardware access is confined to a HAL. Figure 1 shows this structure.

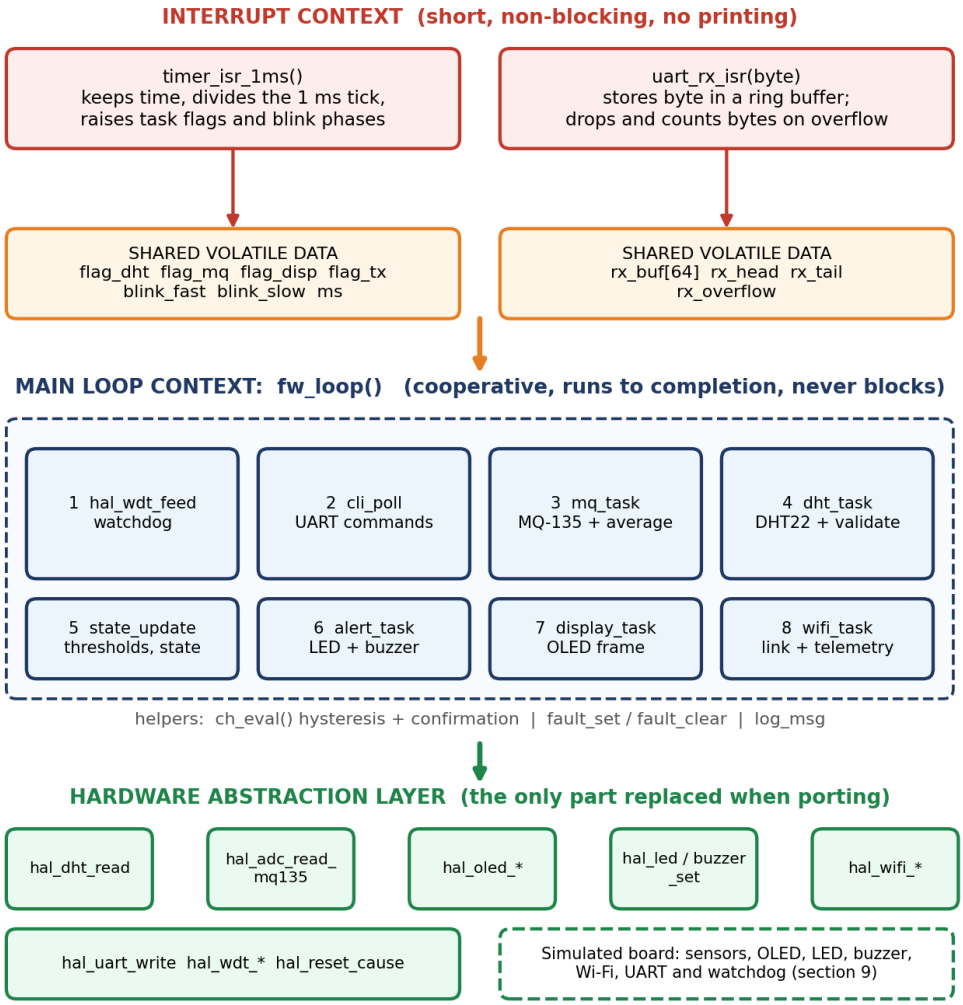


Figure 1: Firmware architecture: interrupt context, shared data, main-loop tasks and HAL

5.1 Modules

Table 4: Firmware modules and key functions

Module (Week 1)	Function(s)	Responsibility
System Initialization	fw_setup()	Sets LED and buzzer OFF first (FR-26), logs the reset cause, initialises thresholds, sensors, OLED, Wi-Fi and watchdog. Failures are recorded as faults, not fatal.
Sensor Acquisition and Processing (DHT22)	dht_task()	Reads the sensor, validates (no response, NaN, range, spike), counts consecutive failures, declares or clears a fault, then evaluates thresholds for valid samples.
Sensor Acquisition and Processing (MQ-135)	mq_task(), mq_average(), aq_update_status()	Reads the ADC, rejects floating or shorted values (0 or full scale), keeps an 8-sample average, captures a clean-air baseline after warm-up and computes a percentage index.
Threshold Monitoring	ch_setup(), ch_eval()	One generic channel type implements trip point, clear point (hysteresis) and confirmation count for all five alert channels.
System state	state_update()	Derives INIT, WARMUP, NORMAL, ALERT or DEGRADED from alerts, warm-up and faults.
Alert Management	alert_task()	Drives LED and buzzer according to the state, with fast and slow patterns and a mute option.
Display Management	build_frame(), display_task()	Builds a six-line frame, writes only when it changes or on a 5 s refresh, detects display failure and retries.
Wi-Fi Communication	wifi_task(), build_and_send()	Non-blocking connection state machine with back-off, periodic telemetry, send-failure handling.
Error Handling	fault_set(), fault_clear(), log_msg()	Fault bit field, transition logging, recovery counting.
Command interface	uart_rx_isr(), cli_poll(), cli_exec(), set_threshold()	Interrupt-fed ring buffer, line assembly, command parsing with input checking.
Main Control Loop	fw_loop()	Calls the tasks in a fixed order each pass: watchdog, UART, MQ-135, DHT22, state, alerts, display, Wi-Fi.

5.2 State Machine

The state model from Week 1 is implemented in state_update(). Alerts take priority over warm-up and faults; faults are shown as DEGRADED when no alert is active. Wi-Fi connectivity is a separate state machine so that losing the network never changes monitoring or alert behaviour.

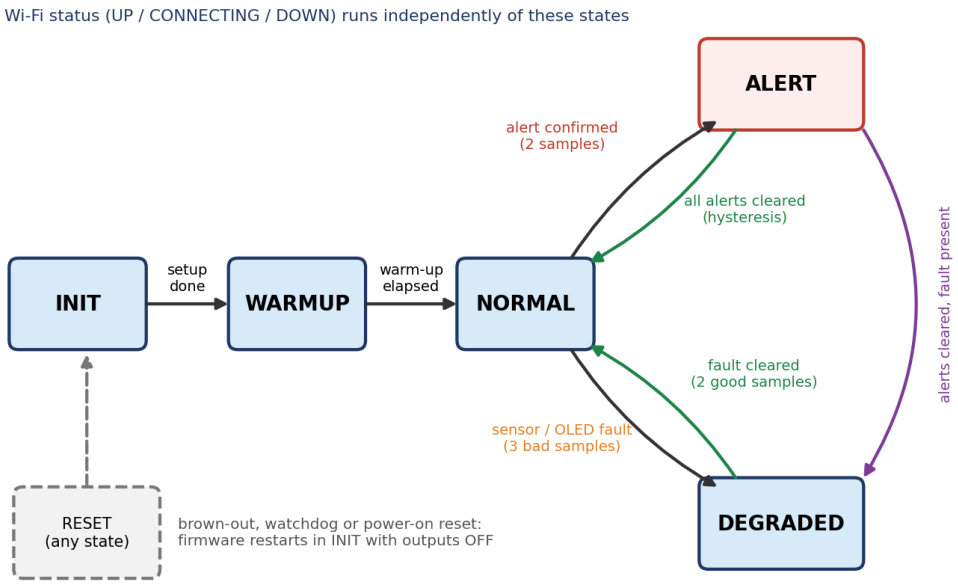


Figure 2: System state machine (temperature and humidity alerts can also occur during WARMUP)

5.3 Shared Data Structures

All firmware state is kept in one structure, `fw_t g`. A reset is therefore a single `memset` followed by `fw_setup()`, which is also how the simulated brown-out and watchdog resets are produced. The structure groups three kinds of data.

Group	Contents
Interrupt-shared (volatile)	ms (system tick), task flags (flag_dht, flag_mq, flag_disp, flag_tx), blink phases, UART ring buffer with head, tail and overflow counter.
Processed data (snapshot_t)	Temperature, humidity and their validity flags; MQ-135 average, percent of baseline, validity and status.
Control state	Alert channels (limit_ch_t), alert and fault bit fields, system state, Wi-Fi state and back-off, display frame, counters and statistics.

5.4 HAL Interface

Table 5: HAL functions (the port boundary)

HAL function	Purpose
hal_dht_read(&t, &h)	Read DHT22; returns false when the sensor does not respond.
hal_adc_read_mq135()	Return a 12-bit ADC value for the MQ-135 channel.
hal_oled_init(), hal_oled_write_frame()	Initialise the display; write six lines; return false on failure.
hal_led_set(), hal_buzzer_set()	Drive the alert outputs.
hal_wifi_begin(), hal_wifi_connected(), hal_wifi_disconnect(), hal_wifi_send(), hal_wifi_rssi()	Non-blocking Wi-Fi control and telemetry transmission.
hal_uart_write()	Transmit diagnostic and command-response text.
hal_wdt_init(), hal_wdt_feed()	Watchdog control.
hal_reset_cause()	Report power-on, brown-out or watchdog reset.

6. Interrupt Design

6.1 Timer Interrupt

`timer_isr_1ms()` represents a hardware timer interrupt that fires every millisecond. It increments the system tick and uses divider counters to raise flags at the periods of the Week 1 timing targets. It contains only increments, comparisons and flag writes: no function calls, no printing and no hardware access, so its execution time is short and bounded. All work is done in the main loop, which takes each flag with `take_flag()`.

Table 6: Timer-derived events

Event	Period	Used by
flag_dht	2000 ms	dht_task(): matches the DHT22 minimum sampling period.
flag_mq	1000 ms	mq_task(): one ADC sample per second feeding the 8-sample average.
flag_disp	1000 ms	display_task(): evaluates whether a new frame is needed.
flag_tx	15000 ms	wifi_task(): telemetry interval.
blink_fast	toggles every 250 ms	alert_task(): buzzer beeping pattern during an alert.
blink_slow	toggles every 1000 ms	alert_task(): LED slow blink during a sensor or display fault.

The flag mechanism avoids doing slow work inside the interrupt and also decouples task timing from how long the main loop takes. On real hardware, reading and clearing a flag must be protected (a critical section or an atomic exchange), because the interrupt can fire between the read and the clear; the source marks the place in `take_flag()`. The single-threaded simulation cannot expose that kind of race, which is listed as a limitation.

6.2 UART Receive Interrupt and Ring Buffer

`uart_rx_isr()` represents the UART receive interrupt. Each received byte is stored in a 64-byte ring buffer at the head index. The main loop reads from the tail in `cli_poll()`. If the buffer is full, the new byte is **dropped and counted** in `rx_overflow` rather than overwriting unread data. `cli_poll()` assembles lines up to 47 characters; a longer line sets an overflow flag and is rejected with `ERR line too long` when its newline arrives, so no buffer is ever overrun.

6.3 Why the Main Loop Never Blocks

There is no `delay()` anywhere in the firmware. Every task returns after a short piece of work, and waiting (Wi-Fi connection timeout, back-off, warm-up, OLED retry) is implemented by comparing the system tick with a stored deadline. This keeps the loop responsive: an alert can be raised, and the watchdog fed, while a Wi-Fi connection attempt is in progress.

7. Key Algorithms

7.1 Validation Pipeline for DHT22 Samples

Stage	Check	Action
1	Sensor responded?	No: the sample is rejected (counted as bad).
2	NaN, or outside -40 to 80 C, or outside 0 to 100 %RH?	Yes: rejected as out of range.
3	Change from last valid sample greater than 10 C or 30 %RH?	Yes: rejected as a spike (physically implausible in 2 s). Skipped while a fault is active so that the sensor can recover.
4	Rejected sample	Last valid value is held. After 3 consecutive rejections a DHT fault is declared, values become invalid (shown as --.- and sent as null) and temperature and humidity alerts are cleared.
5	Valid sample	Resets the failure count, updates the snapshot and evaluates all four temperature and humidity channels. During a fault, 2 consecutive valid samples clear the fault.

7.2 MQ-135 Processing: Relative Index

Week 1 established that the MQ-135 can only give a relative indication. The firmware therefore does not convert counts to ppm. It keeps a moving average of the last 8 valid ADC samples, waits for the warm-up period, captures the averaged value as a **clean-air baseline**, and expresses later readings as a percentage of that baseline. Readings of 0 or full scale (4095) are treated as invalid because they indicate a floating or shorted input.

Index (percent of baseline)	Meaning
Below 125 %	GOOD
125 % up to the alert limit	MODERATE
175 % or more, confirmed twice	Air alert raised; status POOR
Below 160 %	Air alert cleared (hysteresis of 15 percentage points)

These percentages are initial proposals chosen for the simulation. They are defined as constants and will be tuned in Week 3 against realistic data.

7.3 Hysteresis and Confirmation

`ch_eval()` implements all alert channels with one routine. An alert is raised only after **2 consecutive samples** beyond the trip point, which removes isolated bad readings. Once active, it clears only when the value passes a **clear point inside the trip point** (1.0 C for temperature, 2.0 %RH for humidity), which prevents the alert from switching on and off around the limit. The example below uses the default high-temperature limit of 35.0 C (clear point 34.0 C).

Sample (C)	Alert state afterwards	Reason
34.8	Off	Below the trip point; count stays 0.
35.2	Off	First sample beyond the limit; count = 1.
35.4	ON	Second consecutive sample; count = 2 reaches the confirmation limit.
34.6	ON	Inside the hysteresis band; not yet below 34.0.
34.1	ON	Still not below the clear point.
33.9	Off	Below 34.0; the alert clears.

7.4 Wi-Fi State Machine and Back-off

Wi-Fi uses three states: CONNECTING, UP and DOWN (waiting). A connection attempt times out after 5 s. After each failed attempt the firmware waits before retrying, starting at 5 s and doubling up to a limit of 60 s, so a missing network does not cause continuous radio activity. If an established link is lost, monitoring continues and a retry is scheduled after 5 s. Telemetry is attempted only in the UP state and is skipped otherwise. With the access point permanently unavailable the connection attempts start at about 0, 10, 25 and 50 s (measured in simulation, test T12).

7.5 Telemetry Message

The message follows the field list defined in Week 1 and is built with a bounded `snprintf`. If the result would not fit the 256-byte buffer it is not sent. The sequence number increases for every attempted message so that gaps are visible to a receiver. A real example from test T14 (simulated values):

```
{"device_id":"env-node-01","seq":4,"uptime_s":75,"temp_c":25.0,"hum_pct":50.0,
 "aq_raw":798,"aq_status":"GOOD","alert_flags":0,"fault_flags":0,"wifi_rssi_dbm":-61}
```

7.6 Display Update Strategy

The six-line, 21-character frame is built every second but written to the OLED only when its content changes or when a 5 s refresh is due. This reduces I2C traffic while the periodic refresh still detects a disconnected display: after 3 failed writes an OLED fault is raised, monitoring and alerting continue, and the firmware retries initialisation every 30 s. Text longer than a line is truncated explicitly.

8. Configuration Parameters

All tunable values are macros in section 1 of the source file (NFR-03, FR-23). Values marked SIM are chosen for simulation.

Table 7: Main configuration parameters

Parameter	Value	Note
DHT_PERIOD_MS	2000 ms	DHT22 minimum sampling period.
MQ_PERIOD_MS, DISPLAY_PERIOD_MS	1000 ms	
TX_PERIOD_MS	15000 ms	Within the Week 1 target of 15 to 30 s.
MQ_WARMUP_MS	10000 ms (SIM)	Hardware needs several minutes; to be set from the datasheet and observation.
MQ_AVG_LEN	8 samples	Averaging window.
CONFIRM_COUNT	2 samples	Consecutive samples to raise an alert.
FAULT_LIMIT, RECOVER_COUNT	3 and 2	Bad samples to declare a fault; good samples to clear it.
Default thresholds	TEMP_HI 35, TEMP_LO 10, HUM_HI 80, HUM_LO 20	Initial proposals from Week 1, section 7.3.
HYST_TEMP, HYST_HUM	1.0 C, 2.0 %RH	Distance of the clear point from the trip point.

Parameter	Value	Note
AQ_MODERATE / POOR_ON / POOR_OFF	125 / 175 / 160 %	Initial proposals (SIM).
MAX_TEMP_STEP, MAX_HUM_STEP	10 C, 30 %RH	Spike filter limits per sample.
WIFI_CONNECT_TIMEOUT_MS	5000 ms	
WIFI_BACKOFF_MIN / MAX	5 s / 60 s	
OLED_RETRY_MS, OLED_FULL_REFRESH_MS	30 s, 5 s	
WDT_TIMEOUT_MS	10000 ms	Matches the Week 1 target.
UART_RX_BUF_SIZE, UART_LINE_MAX	64, 48	

9. UART Command Interface

The command interface allows thresholds to be inspected and changed at run time without recompiling, and provides a status line for diagnosis. Commands are case-insensitive and end with a newline. Inputs are checked: values must be numbers, within the sensor range, and a high limit must stay above its low limit by more than twice the hysteresis so that the clear zones do not overlap.

Command	Action
HELP	Lists the commands.
STATUS	State, temperature, humidity, air status and percentage, alert and fault flags, Wi-Fi state, uptime, transmissions.
THRESH	Shows the four temperature/humidity limits and the air-quality limit.
SET <TEMP_HI|TEMP_LO|HUM_HI|HUM_LO> <value>	Changes a limit if the value is valid; the alert channel is reset.
MUTE <ON|OFF>	Silences the buzzer; the LED continues to show the alert.

Captured from the demo run (simulated):

```
OK STATE=NORMAL T=33.5 H=50.0 AQ=GOOD(100%) ALERT=0x00 FAULT=0x00 WIFI=UP UP=69s TX=4/4
OK TEMP HI=30.0
[  69027 ms] CONFIG      : TEMP_HI set to 30.0
OK TEMP HI=30.0 TEMP LO=10.0 HUM HI=80.0 HUM LO=20.0 AIR=175%
ERR usage: SET <name> <number>
```

10. Requirement Implementation Matrix

The table links each Week 1 functional requirement to the function that implements it and the test that checks it. Status describes what was verified: simulated tests only, as stated in the scope.

Table 8: Functional requirements: implementation and verification

ID	Implemented in	Behaviour implemented	Test(s)	Result
FR-01	fw_setup()	Start-up initialises GPIO, sensors, OLED, Wi-Fi, watchdog	T01	Pass
FR-02, 03	dht_task()	Temperature and humidity every 2 s	T02	Pass
FR-04	mq_task()	ADC acquisition of MQ-135	T02, T07	Pass
FR-05	dht_task(), mq_task()	Averaging and index computation	T02, T07	Pass
FR-06	dht_task(), mq_task()	Range, spike, rail and missing-data validation	T09, T10, T19	Pass
FR-07	warm_done(), mq_task()	No air alerts until warm-up ends; baseline captured	T01, T08	Pass
FR-08	build_frame()	Temperature and humidity with units on OLED	T02	Pass
FR-09	aq_update_status()	GOOD, MODERATE, POOR, WARMING UP	T02, T07	Pass

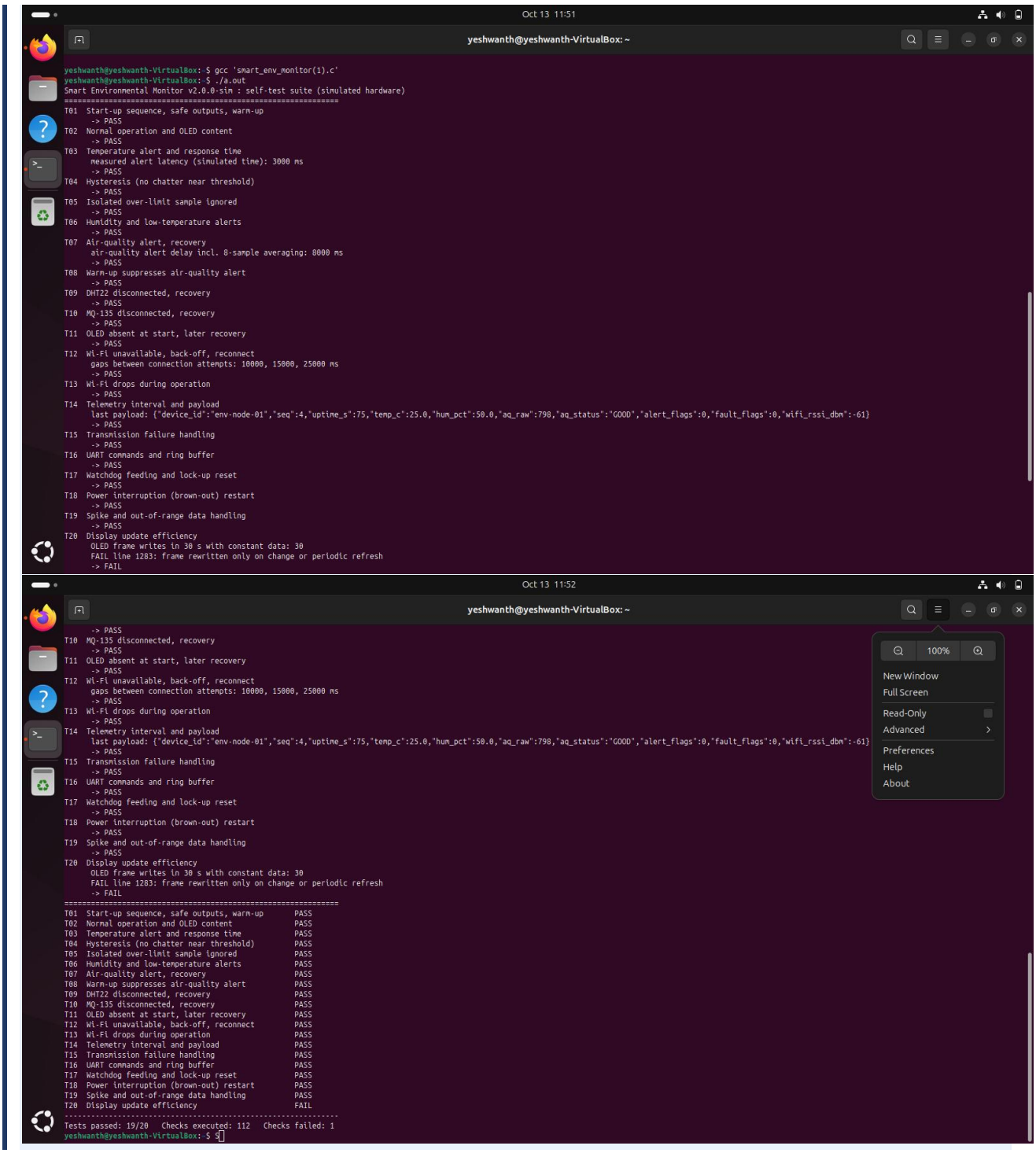
ID	Implemented in	Behaviour implemented	Test(s)	Result
FR-10	build_frame()	Alert and Wi-Fi status on OLED	T02, T13	Pass
FR-11	ch_eval()	Configurable thresholds	T03, T06	Pass
FR-12	ch_eval()	Hysteresis and confirmation	T04, T05, T06	Pass
FR-13, 14	alert_task()	LED and buzzer on and off	T03, T04	Pass
FR-15	alert_task()	Steady LED and beeping for alert; slow blink for fault	T03, T09	Pass
FR-16	wifi_task()	Non-blocking Wi-Fi connection	T01, T12	Pass (simulated)
FR-17, 18	build_and_send()	Periodic JSON with id, sequence, data, flags	T14	Pass (simulated)
FR-19, 20	wifi_task()	Local operation offline; back-off reconnection	T12, T13	Pass (simulated)
FR-21	dht_task(), mq_task()	Sensor fault detection, indication, recovery	T09, T10	Pass
FR-22	display_task()	Monitoring continues without OLED; retry	T11	Pass
FR-23	set_threshold(), cli_exec()	Thresholds as named constants and run-time SET command	T16	Pass (no non-volatile storage)
FR-24	log_msg(), cli_exec()	Diagnostic log and STATUS	T16, demo	Pass
FR-25	hal_wdt_*, fw_loop()	Watchdog fed every loop; reset on lock-up	T17	Pass (simulated)
FR-26	fw_setup()	Outputs OFF at every start	T01, T17, T18	Pass

Non-functional requirements addressed by the implementation: NFR-02 (fault isolation, tests T09 to T11), NFR-03 (central configuration and modules), NFR-07 (alert response, T03), NFR-09 and NFR-10 (HAL separation). NFR-08 (power), NFR-12 (security, real encrypted transport) and NFR-13 (environment) require real hardware or network and are not verified in Week 2.

11. Test Strategy

The test strategy is to verify every Week 1 requirement and every row of the Week 1 fault table by injecting the corresponding condition into the simulated board and observing the firmware's outputs.

- **Deterministic virtual time.** Each simulated millisecond calls the timer interrupt and one pass of the main loop. A test can run minutes of device time instantly and repeatably.
- **Stimulus injection.** Tests set temperature, humidity, ADC level, sensor failure, OLED presence, access-point availability, send success and lock-up.
- **Observation.** Tests read the LED and buzzer state and time, the OLED lines, the transmitted messages and timing, the UART output, and internal state (alerts, faults, state).
- **Assertions.** 112 checks (CHECK macro) across 20 tests. Each test reports PASS or FAIL and the program exit code reflects the overall result.
- **Physically plausible stimuli.** Temperature and humidity are changed in steps that a real room could produce (helpers ramp_temp and ramp_hum), because the firmware rejects implausible jumps (see defect D1).
- **Robustness runs.** The same tests and the demo were also run under AddressSanitizer and UndefinedBehaviorSanitizer.



12. Test Cases and Results

The table below summarises the 20 tests as run. The result column is copied from the final execution of the self-test suite (see Appendix A for the raw output).

Table 9: Test cases and results (simulated hardware)

ID	Test	Scenario and what is checked	Result
T01	Start-up and warm-up	Boot: outputs OFF, state WARMUP, air status WARMING UP on the OLED; after warm-up state NORMAL and baseline captured.	PASS
T02	Normal operation	25.0 C, 50 %RH, baseline air: values acquired and shown on the OLED, Wi-Fi OK, alert NONE, outputs OFF.	PASS

ID	Test	Scenario and what is checked	Result
T03	Temperature alert	Temperature rises to 36 C: alert within two sampling periods, LED on, buzzer sounds, state ALERT, OLED shows TEMP_HI.	PASS
T04	Hysteresis	Alert stays at 34.5 C (inside the band), clears at 33.5 C, outputs OFF.	PASS
T05	Isolated sample	One over-limit sample then normal: no alert and no fault.	PASS
T06	Humidity and low temperature	85 %RH raises HUM_HI, stays at 79, clears at 77; 8 C raises TEMP_LO.	PASS
T07	Air-quality alert	Reading doubles relative to baseline: AIR alert, status POOR; returns to GOOD when air recovers.	PASS
T08	Warm-up suppression	High ADC value during warm-up: no alert, WARMING UP; baseline taken afterwards, no false alert.	PASS
T09	DHT22 disconnected	Fault after 3 failed reads, state DEGRADED, --.- and DHT on OLED, no alert, MQ-135 continues, LED slow blink, buzzer silent; fault clears after recovery.	PASS
T10	MQ-135 disconnected	ADC stuck at 0: fault, air status unknown, temperature alert still works; recovers to GOOD.	PASS
T11	OLED absent	Fault at start; alert, LED and telemetry still work; display recovered by the 30 s retry.	PASS
T12	Wi-Fi unavailable	120 s offline: retries with growing gaps, sensing and alerting continue; reconnects within 70 s of the network returning.	PASS
T13	Wi-Fi drop	Link loss detected, OLED shows status, sampling continues, automatic reconnection.	PASS
T14	Telemetry	At least 4 messages at 15 s intervals, sequence +1 per message, required fields present, temp_c null during DHT fault.	PASS
T15	Send failure	Failed sends counted, TX fault flagged, link stays up, fault clears after a successful send.	PASS
T16	UART interface	STATUS, THRESH, SET, MUTE work; bad number, out-of-range value, unknown command, over-long line and ring-buffer overflow are handled; interpreter works afterwards.	PASS
T17	Watchdog	Fed regularly (max gap well below the timeout); after a simulated 10.5 s lock-up the firmware is reset, cause WATCHDOG logged, outputs OFF.	PASS
T18	Brown-out restart	Restart during an alert: outputs OFF at once, WARMUP, uptime restarted, cause logged, alert re-evaluated from fresh data.	PASS
T19	Spike and range	A 13 C one-sample spike is rejected without fault; persistent 120 C becomes a fault with no alert; recovery on valid data.	PASS
T20	Display efficiency	Constant data for 30 s: frame written only on the 5 s refresh.	PASS

12.1 Values Observed in the Simulation

These values were printed by the tests. They are in **simulated time** and show the firmware logic behaving as designed. They are not measurements on hardware.

Observation	Value	Comment
Temperature alert delay after step to 36 C (T03)	3000 ms	Within the 4000 ms bound (two 2 s sampling periods).
Air-quality alert delay at 2x baseline (T07)	8000 ms	Includes 8-sample averaging and confirmation; slower by design.
Wi-Fi attempt start gaps with network down (T12)	10000, 15000, 25000 ms	Increasing, as designed (5 s timeout plus doubling back-off).
Telemetry interval (T14)	15 s	Checked within 5 ms for every message.
OLED frame writes in 30 s with constant data (T20)	6	Periodic refresh only.
Total checks executed / failed	112 / 0	Final run.

The air-quality delay is worth noting for Week 3. The Week 1 target (alert within two sampling periods of a confirmed crossing) holds for temperature and humidity. For air quality the averaging window adds delay before the processed value crosses the limit. This is a deliberate trade-off against noise, and the window length can be revisited with real sensor data.

12.2 Example Scenario Log

The scripted demo (`./smart_env_monitor demo`) shows a complete scenario with the OLED frame after each change. Part of its log is shown below; the full output is in demo_output.txt.

```
>>> [     0 ms] SCENARIO: power on, warm-up, normal operation
[      0 ms] BOOT        : env-node-01 v2.0.0-sim, reset cause POWER-ON
[      0 ms] FAULT SET   : Wi-Fi (offline)
[      0 ms] BOOT        : initialisation complete, MQ-135 warm-up started
[   1500 ms] FAULT CLEAR : Wi-Fi (offline)
[   1500 ms] WIFI        : connected (RSSI -61 dBm)
[  10000 ms] MQ-135 baseline captured: 800 counts
[  10000 ms] STATE       : WARMUP -> NORMAL
>>> [ 13000 ms] SCENARIO: temperature rises to 36 C (gradual ramp)
[  20000 ms] ALERT ON    : TEMP_HI (value 36.0, limit 35.0)
[  20000 ms] STATE       : NORMAL -> ALERT
>>> [ 23000 ms] SCENARIO: temperature falls to 34.5 C (inside hysteresis band): alert must remain
>>> [ 29000 ms] SCENARIO: temperature falls to 33.5 C: alert clears
[  30000 ms] ALERT OFF   : TEMP_HI (value 33.5, cleared at 34.0)
[  30000 ms] STATE       : ALERT -> NORMAL
>>> [ 35000 ms] SCENARIO: DHT22 unplugged
[  36000 ms] DHT22 sample rejected (no response), consecutive=1
[  38000 ms] DHT22 sample rejected (no response), consecutive=2
[  40000 ms] DHT22 sample rejected (no response), consecutive=3
[  40000 ms] FAULT SET   : DHT22 (sensor)
[  40000 ms] STATE       : NORMAL -> DEGRADED
[  42000 ms] DHT22 sample rejected (no response), consecutive=4
>>> [ 43000 ms] SCENARIO: DHT22 reconnected
[  46000 ms] FAULT CLEAR : DHT22 (sensor)
[  46000 ms] STATE       : DEGRADED -> NORMAL
>>> [ 49000 ms] SCENARIO: Wi-Fi access point lost, then restored
[  49001 ms] FAULT SET   : Wi-Fi (offline)
[  49001 ms] WIFI        : link lost, offline mode
[  57001 ms] FAULT CLEAR : Wi-Fi (offline)
[  57001 ms] WIFI        : connected (RSSI -61 dBm)
>>> [ 69000 ms] SCENARIO: UART commands
[  69027 ms] CONFIG      : TEMP HI set to 30.0
>>> [ 69065 ms] SCENARIO: end of demo
```

One OLED frame from the demo while the alert is active (simulated display):

```
+--------------------+
| ENV MONITOR WiFi:OK  |
| Temp : 36.0 C        |
| Hum  : 50.0 %        |
| Air  : GOOD          |
| Alert: TEMP HI       |
| State: ALERT         |
+--------------------+
```

13. Debugging and Defect Log

The first complete run of the suite passed 12 of 20 tests (17 failed checks). The failures were analysed one by one. Most were caused by unrealistic test stimuli, not firmware faults, but they exposed behaviour worth recording. The suite then reached 20 of 20.

Table 10: Defects and observations found during Week 2 testing

ID	Symptom	Root cause	Fix	Outcome
D1	T03, T04, T06, T10, T11, T12 failed: no alert after a step to 36 C or 85 %RH.	The test applied an instant 11 C or 35 %RH step. The spike filter (limits 10 C and 30 %RH per sample) rejected it as implausible, which is correct behaviour.	Test stimuli changed to realistic ramps (ramp_temp, ramp_hum). Firmware unchanged.	Fixed. The step limits should be reviewed with real sensor data in Week 3.
D2	T16: over-long line and the command after the overflow test were not handled.	The simulator delivered a long command instantly; the 64-byte ring buffer dropped bytes including the newline, so the next line was merged with junk.	Simulator now delivers about 1 byte per ms (roughly 9600 baud). Overflow is tested explicitly, followed by a resynchronising newline.	Fixed. Behaviour noted: an overflow that swallows a newline discards the following line.
D3	T09: LED slow-blink check failed.	The check window overlapped the 6 s needed to declare the fault (3 reads at 2 s), so too few toggles were counted.	Window moved to start after the fault is declared.	Fixed (test error).

ID	Symptom	Root cause	Fix	Outcome
D4	Compiler warnings (-Wformat-truncation) in build_frame().	OLED text such as a long alert list or fault list could exceed 21 columns. A first fix limited text to 15 characters, which still exceeded the width by one; the compiler caught this at -O0.	Explicit precision in the format strings (14 characters after the 7-character label).	Fixed. Builds are warning-free.

These steps illustrate the intended workflow: run, observe, find the root cause, fix the right component (test or firmware), and re-run. Week 3 will extend this with real hardware behaviour and performance measurements.

14. Code Quality and Documentation

Aspect	What was done
Single file organisation	Numbered sections: configuration, types and state, HAL interface, logging, interrupts, modules, entry points, simulated HAL, tests, demo and main. An embedded README at the top explains purpose, compile and run steps, commands and porting.
Inline comments	Every section and non-obvious decision is commented, including why rules exist (for example the ISR rules, the spike filter being skipped during a fault, the baseline approach for the MQ-135).
Naming and style	Descriptive names, module-prefixed functions (dht_, mq_, ch_, hal_, sim_), constants for every tunable value, no magic numbers in logic.
Memory use	No dynamic allocation. Fixed-size buffers; formatted output uses bounded snprintf and explicit truncation.
Robustness	Inputs validated (sensor data, commands); overflow detected and counted; every failure path leaves the loop running.
Compiler checks	Zero warnings with -Wall -Wextra -Wpedantic at -O0 and -O2.
Sanitizer runs	Tests and demo run with AddressSanitizer and UndefinedBehaviorSanitizer: no reports.
Size	About 1,380 lines including comments, the simulated HAL, the 20 tests and the demo.

No claim is made of compliance with a formal coding standard such as MISRA C.

15. How to Compile and Run

Requirements: a C99 compiler (GCC or Clang). No other libraries.

```
# compile
gcc -std=c99 -Wall -Wextra -Wpedantic -O2 -o smart_env_monitor smart_env_monitor.c -lm

# run
./smart_env_monitor            # 20-test self-test suite
./smart_env_monitor verbose    # tests plus the firmware UART log
./smart_env_monitor demo       # scripted scenario with OLED frames
./smart_env_monitor help
```

On Windows with MinGW use the same command; the program is `smart_env_monitor.exe`. Redirect output to a file to keep a record, for example `./smart_env_monitor > test_results.txt`. The exit code is 0 when every test passes.

16. Porting to ESP32 Hardware

Sections 1 to 7 of the source are hardware independent. Only the simulated HAL (section 9) must be replaced. The mapping below is a **proposal** based on the Week 1 pin map. It has **not been implemented or tested** and is the starting point for hardware work in Weeks 3 and 4.

Table 11: Proposed HAL mapping for ESP32 (not tested)

HAL function	Proposed ESP32 implementation
hal_dht_read	DHT library on GPIO4 (10 kOhm pull-up); read no faster than every 2 s; treat NaN as no response.

HAL function	Proposed ESP32 implementation
hal_adc_read_mq135	analogRead on ADC1 GPIO34 through the voltage divider (ADC2 is unusable with Wi-Fi); consider oversampling.
hal_oled_init, hal_oled_write_frame	SSD1306 driver (for example U8g2 or Adafruit SSD1306) on I2C GPIO21 and GPIO22; return false when the device does not acknowledge.
hal_led_set, hal_buzzer_set	digitalWrite on GPIO25 (LED through a resistor) and GPIO26 (buzzer transistor driver).
hal_wifi_*	WiFi station mode, status polling, and an HTTP or MQTT client with explicit timeouts; RSSI from the Wi-Fi API.
hal_uart_write, uart_rx_isr	Serial output; receive through the UART interrupt or driver callback feeding the same ring buffer.
hal_wdt_init, hal_wdt_feed	ESP32 task watchdog with a 10 s timeout.
hal_reset_cause	Reset-reason function of the ESP32 SDK.
timer_isr_1ms	Hardware timer interrupt at 1 ms. The ISR must stay as short as it is now; flag handling in take_flag() needs a critical section or atomic access.

A real DHT22 returns data only after power-up settling and may need a longer first-read delay. The first DHT read in this firmware is at 2 s, which is consistent with that.

17. Limitations and Known Issues

- **Simulation fidelity.** Results demonstrate the firmware logic, not real hardware timing, sensor behaviour, Wi-Fi performance or power use.
- **Interrupt concurrency.** The simulation runs in one thread, so races between the ISR and the main loop (flag handling, ring buffer indices) cannot be observed. They must be reviewed and tested on hardware.
- **MQ-135 model.** The simulated sensor is a constant plus small noise. Selectivity, drift, warm-up and humidity dependence are not modelled. The 10 s warm-up is a simulation value.
- **Placeholder values.** Thresholds, air-quality percentages and spike-filter limits are initial proposals, not calibrated values.
- **UART overflow.** When the receive buffer overflows, the dropped bytes can include a newline, so the following command line is discarded. This is acceptable for a diagnostic interface but would need flow control or a larger buffer for critical commands.
- **Configuration persistence.** Thresholds changed over UART are lost at reset; non-volatile storage is future work.
- **Network and security.** The Wi-Fi layer is simulated. Real HTTPS or MQTT with TLS, authentication and credential handling is not implemented.
- **Telemetry buffering.** Messages are skipped, not queued, while offline.

18. Week 2 Deliverables

Deliverable	Content	Status
smart_env_monitor.c	Complete firmware, simulated HAL, 20 self-tests, demo, inline comments and embedded README (single file)	Done
README.md	Description, compile and run steps, commands, key functions, porting and roadmap	Done
test_results.txt	Captured output of the self-test suite	Done
demo_output.txt	Captured output of the scripted demo	Done
Week 2 technical report (this document)	Architecture, algorithms, requirement mapping, test process and results, defect log, limitations	Done
Screenshot of your own run	Terminal showing compile and 20/20 result	To be captured by you

19. Preparation for Weeks 3 and 4

The following are planned activities that build on Week 2. None is reported as completed.

Week	Planned focus, based on Week 2 findings
Week 3: Debugging and Performance Optimization	Port the HAL to ESP32 or an equivalent simulator environment; measure real loop timing, interrupt behaviour and current; review the spike-filter limits and averaging window with real sensor data; tune thresholds and the MQ-135 warm-up; review interrupt/main-loop data sharing for races; examine ring-buffer sizing.
Week 4: System Integration and Final Testing	Integrate sensors, OLED, alert hardware and Wi-Fi; test against the Week 1 requirements and the traceability matrix; run an extended unattended test; document final results and remaining limitations.

20. Conclusion

Week 2 implemented the Week 1 design as a single-file C firmware with a timer interrupt, a UART receive interrupt with a ring buffer, a non-blocking cooperative scheduler, validated and filtered sensor processing, hysteresis-based alerts, a state machine, an OLED manager, a Wi-Fi manager with back-off, a command interface, and fault handling for each fault identified in Week 1. A hardware abstraction layer keeps the logic independent of the board.

Because no hardware was used, the firmware was tested against a simulated board with fault injection. A suite of 20 tests with 112 checks ran in virtual time and passed after the defects in Section 13 were resolved, and the code builds without warnings and ran cleanly under sanitizers. These results show that the logic meets the Week 1 requirements in simulation; they do not yet show behaviour on real hardware.

The next steps are to run the same firmware logic on hardware through a real HAL (Week 3) and to integrate and test the complete system (Week 4), using the limitations and observations recorded here as the starting checklist.

Appendix A: Self-Test Output (test_results.txt)

Raw output of `./smart_env_monitor` from the final run (simulated hardware).

```
Smart Environmental Monitor v2.0.0-sim : self-test suite (simulated hardware)
============================================================
T01  Start-up sequence, safe outputs, warm-up
      -> PASS
T02  Normal operation and OLED content
      -> PASS
T03  Temperature alert and response time
       measured alert latency (simulated time): 3000 ms
      -> PASS
T04  Hysteresis (no chatter near threshold)
      -> PASS
T05  Isolated over-limit sample ignored
      -> PASS
T06  Humidity and low-temperature alerts
      -> PASS
T07  Air-quality alert, recovery
       air-quality alert delay incl. 8-sample averaging: 8000 ms
      -> PASS
T08  Warm-up suppresses air-quality alert
      -> PASS
T09  DHT22 disconnected, recovery
      -> PASS
T10  MQ-135 disconnected, recovery
      -> PASS
T11  OLED absent at start, later recovery
      -> PASS
T12  Wi-Fi unavailable, back-off, reconnect
       gaps between connection attempts: 10000, 15000, 25000 ms
      -> PASS
T13  Wi-Fi drops during operation
      -> PASS
T14  Telemetry interval and payload
       last payload: {"device_id":"env-node-
01","seq":4,"uptime_s":75,"temp_c":25.0,"hum_pct":50.0,"aq_raw":798,"aq_statu
      -> PASS
T15  Transmission failure handling
      -> PASS
T16  UART commands and ring buffer
      -> PASS
T17  Watchdog feeding and lock-up reset
      -> PASS
T18  Power interruption (brown-out) restart
      -> PASS
T19  Spike and out-of-range data handling
      -> PASS
T20  Display update efficiency
       OLED frame writes in 30 s with constant data: 6
      -> PASS
============================================================
T01  Start-up sequence, safe outputs, warm-up       PASS
T02  Normal operation and OLED content              PASS
T03  Temperature alert and response time            PASS
T04  Hysteresis (no chatter near threshold)         PASS
T05  Isolated over-limit sample ignored             PASS
T06  Humidity and low-temperature alerts            PASS
T07  Air-quality alert, recovery                    PASS
T08  Warm-up suppresses air-quality alert           PASS
T09  DHT22 disconnected, recovery                   PASS
T10  MQ-135 disconnected, recovery                  PASS
T11  OLED absent at start, later recovery           PASS
T12  Wi-Fi unavailable, back-off, reconnect         PASS
T13  Wi-Fi drops during operation                   PASS
T14  Telemetry interval and payload                 PASS
T15  Transmission failure handling                  PASS
T16  UART commands and ring buffer                  PASS
T17  Watchdog feeding and lock-up reset             PASS
T18  Power interruption (brown-out) restart         PASS
T19  Spike and out-of-range data handling           PASS
T20  Display update efficiency                      PASS
------------------------------------------------------------
Tests passed: 20/20   Checks executed: 112   Checks failed: 0
```

Appendix B: Source File Map

Section	Title	Contents
Header	Embedded README	Purpose, features, compile and run, commands, file map, porting notes.
1	Configuration	All tunable constants.
2	Types and state	Enumerations, limit channel, snapshot, firmware state object.
3	HAL interface	Declarations of every hardware function (port boundary).
4	Logging and helpers	uart_print, log_msg, take_flag, fault handling, formatting.

Section	Title	Contents
5	Interrupt routines	timer_isr_1ms, uart_rx_isr.
6	Firmware modules	Thresholds, DHT22, MQ-135, state, alerts, display, Wi-Fi, command interface.
7	Entry points	fw_setup, fw_loop.
9	Simulated HAL	Simulated board and simulator controls.
10	Self-test suite	20 tests and the runner.
11	Demo and main	Scripted demo and program entry.